📘 Guide de démarrage : cloner le projet et faire fonctionner la recherche

Ce guide part d'un `git clone` tout neuf et arrive à un moteur de recherche qui répond dans le navigateur. Il est découpé en **niveaux** : chacun ajoute une fonction et se teste seul, vous pouvez vous arrêter après n'importe lequel.

Niveau	Ce que vous obtenez	Durée environ	Docker	Téléchargement
1	Recherche par image dans la base **Objects** (couleur, texture, forme) + mots-clés (noms de classes)	15 min	non	aucun (archive de l'enseignant)
2	+ descripteur profond **VGG16** (nettement plus précis)	30 min de calcul	non	530 Mo
3	+ mots-clés par **Elasticsearch**, chargés par **Logstash**	10 min	oui	3,4 Go d'images Docker
4	+ base **MIRFLICKR-1M** (1 000 000 d'images, descripteurs MPEG-7, vignettes)	1 h 30	oui	4,0 Go

> **Mesures** faites sur notre machine (16 cœurs, 16 Go de RAM, Windows 11) : index Objects HSV/LBP/HOG en 40 s, VGG16 en 28 min, import des 1 000 000 de signatures en 5 min, téléchargement MIRFLICKR en 67 min. Vos durées dépendent de votre machine et de votre connexion.

---

0. Ce qu'il faut avant de commencer

Outil	Version	Pourquoi
Git	récent	cloner le dépôt
Python	**3.10, 3.11 ou 3.12**	TensorFlow (VGG16) ne fonctionne **pas** avec Python 3.13 / 3.14
Docker Desktop	récent (niveaux 3 et 4)	Elasticsearch et Logstash
Espace disque	niveau 1 : 1 Go ; niveau 4 : environ 10 Go	images, signatures, archives, images Docker
Mémoire	8 Go conseillés (niveau 4)	la matrice de signatures du million d'images est chargée en mémoire

Vérifiez votre Python :

```
python --version       # doit afficher 3.10.x, 3.11.x ou 3.12.x
```

(Sous Windows, `py -3.12 --version` permet de choisir une version précise ; installation : `winget install Python.Python.3.12`.)

1. Niveau 1 : la base Objects, sans Docker

1.1 Cloner et installer

```
git clone <URL-DU-DEPOT> cbir
cd cbir
python -m venv .venv                  # sous Windows : py -3.12 -m venv .venv
```

Activez l'environnement :

```
source .venv/bin/activate             # Linux / macOS
.venv\Scripts\activate                # Windows (PowerShell ou cmd)
```

> Si l'activation est bloquée sous PowerShell, n'activez rien : remplacez `python` par `.venv\Scripts\python` dans toutes les commandes suivantes.

```
pip install -r requirements.txt       # 5 à 10 min (TensorFlow est gros)
```

1.2 Mettre les images Objects au bon endroit

L'archive `idb.zip` fournie par l'enseignant contient deux dossiers : `Objects/` (images classées, un sous-dossier par classe) et `Texture/`. Dézippez-la dans `data/` :

```
python -m zipfile -e chemin/vers/idb.zip data/
```

Résultat attendu :

```
cbir/
└── data/
    ├── Objects/
    │   ├── accordion/image_0001.jpg ...
    │   ├── airplanes/ ...
    │   └── ...            (environ 9 100 images, 102 dossiers)
    └── Texture/           (non utilisé par le moteur)
```

> Pas d'archive ? `python scripts/make_demo_data.py --out data/Objects` crée une petite base synthétique de test (10 classes).

1.3 Calculer les signatures (phase hors ligne)

```
python scripts/build_index.py --root data/Objects --desc hsv lbp hog
```

Cela lit chaque image et écrit `index/hsv.npy`, `index/lbp.npy`, `index/hog.npy` et `index/meta.json` (environ 40 s pour 9 145 images). Dernière ligne attendue : `[index] done in ...`.

1.4 Lancer le serveur

```
python -m uvicorn server:app --port 8000
```

Ouvrez **http://localhost:8000**. Dans la console, vous devez lire :

```
[server] base objects: 9145 images, descriptors ['hsv', 'lbp', 'hog'], keywords via in-process BM25
```

1.5 Premier test

1. Dans « Pick a query from the base », **cliquez sur une image** : 20 résultats apparaissent, les plus proches d'abord, avec leur distance.
2. Cliquez sur un résultat : la recherche repart de cette image.
3. Tapez `butterfly` dans **Keywords** et cliquez **Search** : les images de la classe butterfly.
4. Essayez de décocher un descripteur ou de changer les poids (curseurs).

Test en ligne de commande :

```
curl -X POST localhost:8000/api/search -F image_id=10 -F descs=hsv,lbp -F k=3
```

✅ **Niveau 1 terminé.** Les niveaux suivants s'ajoutent sans rien casser.

---

2. Niveau 2 : ajouter VGG16 (optionnel)

VGG16 est un réseau de neurones qui décrit le **contenu** de l'image. Sur nos mesures, il est de loin le meilleur descripteur (précision moyenne 0,77 contre 0,29 à 0,43 pour les trois autres).

```
python scripts/build_index.py --root data/Objects --desc hsv lbp hog vgg
```

- Au premier lancement, Keras **télécharge les poids ImageNet (environ 530 Mo)** depuis les serveurs de Google, dans `~/.keras/models` ; ils servent ensuite hors ligne.
- Le calcul des 9 145 images prend environ 30 min sur CPU (batches de 32). Il recalcule aussi HSV, LBP, HOG.
- Relancez le serveur : la case **Deep features** apparaît dans l'interface. Cochez-la seule pour comparer.

---

3. Niveau 3 : Elasticsearch et Logstash

Ils stockent les **mots-clés et les informations de chaque image** (chemin, étiquette, tags) et répondent aux recherches par mots-clés (BM25, recherche floue). Les signatures restent dans les fichiers `.npy`.

3.1 Démarrer Elasticsearch

Lancez **Docker Desktop** et attendez « Engine running », puis :

```
docker compose up -d elasticsearch        # 1re fois : télécharge l'image (environ 2 Go)
curl localhost:9200                        # doit répondre en JSON (cluster_name, version 8.15.3)
```

3.2 Créer les index et charger les données avec Logstash

```
python scripts/create_es_indices.py                 # crée les index « objects » et « mirflickr
»
python scripts/make_objects_csv.py --index index    # écrit index/objects.csv
docker compose run --rm logstash -f /usr/share/logstash/pipeline/objects.conf
```

> **Git Bash sous Windows** : préfixez la dernière commande par `MSYS_NO_PATHCONV=1`, sinon le chemin `/usr/share/...` est réécrit et Logstash répond « No config files found ». PowerShell n'a pas ce problème.

La commande se termine quand le fichier est lu (`Pipeline terminated` puis `Logstash shut down`).

3.3 Vérifier

```
curl "localhost:9200/_cat/indices?v"       # objects : 9145 documents
```

Relancez le serveur : la ligne de démarrage doit maintenant dire `keywords via Elasticsearch`, et l'interface affiche « keywords by Elasticsearch » sous les résultats.

---

4. Niveau 4 : la base MIRFLICKR-1M

Nous n'utilisons **pas** les images originales (10 archives de 12 Go) : la recherche travaille sur les signatures MPEG-7 fournies, et seules les vignettes 64×64 servent à l'affichage.

4.1 Télécharger (environ 4,0 Go)

```
python scripts/download_mirflickr.py --dry-run     # liste les fichiers et leurs tailles, sans
rien télécharger
python scripts/download_mirflickr.py               # télécharge tout dans data/mirflickr_raw/
(reprise possible)
```

Fichier	Taille	Rôle
`features_edgehistogram.zip`	595 Mo	descripteur MPEG-7 EHD
`features_homogeneoustexture.zip`	344 Mo	descripteur MPEG-7 HTD
`tags.zip`	219 Mo	mots-clés
`thumbs1-33.zip`, `thumbs34-66.zip`, `thumbs67-100.zip`	901 + 888 + 911 Mo	vignettes (un tiers de la base chacune)

Moins de place ou de temps ? `--only ehd htd tags thumbs1` télécharge les signatures, les tags et le premier tiers des vignettes. La recherche fonctionne sur le million d'images ; les images des deux autres tiers s'affichent simplement en gris (vignette absente).

4.2 Construire l'index (environ 5 min)

```
python scripts/import_mirflickr.py --raw data/mirflickr_raw --index index_1m
```

Les archives sont lues **sans extraction**. Résultat : `index_1m/ehd.npy` (1 000 000 × 150), `index_1m/htd.npy` (1 000 000 × 43), `index_1m/tags.csv` et `index_1m/meta.json`. Pour un essai rapide : ajoutez `--limit 20000 --index index_20k`.

4.3 Charger les tags dans Elasticsearch

```
docker compose up -d elasticsearch                # s'il n'est pas déjà lancé
python scripts/create_es_indices.py               # s'il n'a pas été fait au niveau 3
docker compose run --rm logstash -f /usr/share/logstash/pipeline/mirflickr.conf      # environ 1 à 2 min
curl "localhost:9200/_cat/indices?v"              # mirflickr : 946113 documents
```

(Les 53 887 images sans aucun tag ne sont pas indexées dans Elasticsearch : Logstash refuse les lignes CSV au champ vide ; elles restent trouvables par leur contenu visuel.)

4.4 Lancer et utiliser

```
python -m uvicorn server:app --port 8000
```

Le démarrage affiche les deux bases :

```
[server] base objects: 9145 images, descriptors ['hsv', 'lbp', 'hog', 'vgg'], keywords via Elasticsearch
[server] base mirflickr: 1000000 images, descriptors ['ehd', 'htd'], keywords via Elasticsearch
```

Dans l'interface :

- Le menu **Image base** choisit la base. Ticker un descripteur **grise** ceux de l'autre base : un descripteur appartient à une seule base, on ne peut pas les mélanger.
- **EHD / HTD** (MIRFLICKR) : cliquez une vignette. La **première recherche** construit l'arbre (10 à 25 s pour EHD selon la machine) ; les suivantes durent moins d'une seconde.

- Pas d'envoi de fichier sur MIRFLICKR (les signatures MPEG-7 sont déjà calculées) : cherchez à partir d'une image de la base.
- **Keywords** : `sunglasses` (base MIRFLICKR) renvoie 1 184 images.
- **Recherche floue** (tolérance aux fautes) : disponible dans l'API, sur la base Objects : `curl -X POST localhost:8000/search_by_text -H "Content-Type: application/json" -d '{"tags":"buterfly","type":"fuzzy","number":3}'` trouve les 91 images « butterfly » malgré la faute.

---

5. Vérifier que tout fonctionne

Test automatique de la recherche (serveur lancé dans un autre terminal) :

```
python scripts/smoke_test.py
```

Il enchaîne 13 vérifications (recherche par image, par envoi de fichier, par mots-clés, ET image + mots-clés, recherche floue, base MIRFLICKR, refus du mélange HOG + EHD…), affiche `PASS` / `FAIL` pour chacune et se termine par `13/13 checks passed`. Les bases non chargées sont ignorées (`[skip]`), pas comptées en échec.

Autres vérifications :

```
python -m pytest -q tests                 # 23 tests unitaires, sans Elasticsearch ni base
réelle
curl localhost:8000/api/info              # les bases chargées, leurs descripteurs et leurs
classes
```

> **PowerShell** : `curl` y est un alias de `Invoke-WebRequest` . Écrivez `curl.exe` pour les commandes de ce guide (ou utilisez Git Bash).

Mesures de qualité et de vitesse (rapport) :

```
python scripts/evaluate.py --queries 200 --out results    # précision / rappel / VP-tree sur
Objects
python rapport/collect_data.py                             # VP-tree contre exhaustif sur 1 000
000 d'images
```

Documentation interactive de l'API : **http://localhost:8000/docs**.

---

6. Problèmes fréquents

Symptôme	Cause et solution
`No matching distribution found for tensorflow`	Python 3.13 / 3.14. Installez 3.12 et recréez `.venv`.
`No index found. Run: python scripts/build_index.py ...`	Le serveur ne trouve ni `index/meta.json` ni `index_1m/meta.json` : refaites le §1.3.
`connection refused` sur le port 9200	Elasticsearch n'est pas lancé : `docker compose up -d elasticsearch`. Le serveur marche quand même (mots-clés en mémoire pour Objects ; **pas** de mots-clés pour MIRFLICKR).
Logstash : `No config files found in path ... C:/Program Files/Git/...`	Git Bash a réécrit le chemin : préfixez par `MSYS_NO_PATHCONV=1` ou utilisez PowerShell.
`docker compose` : `cannot connect to the Docker daemon`	Docker Desktop n'est pas démarré.
`address already in use` (port 8000)	Un autre serveur tourne : arrêtez-le ou ajoutez `--port 8001`.
Les vignettes MIRFLICKR sont grises	Le zip des vignettes correspondant n'est pas dans `data/mirflickr_raw/` (voir §4.1), ou le serveur n'a pas été relancé après le téléchargement.
La page affiche une ancienne version	Rechargement forcé du navigateur (Ctrl + F5).
Premier appel lent (10 à 25 s)	Normal : construction du VP-tree pour cette combinaison de descripteurs.
`docker compose up` : `elasticsearch` s'arrête (code 137 / 78)	Mémoire Docker insuffisante : allouez 4 Go à Docker Desktop (Settings, Resources), ou sous Linux `sysctl -w vm.max_map_count=262144`.

Tout remettre à zéro (les données du dépôt ne sont pas touchées) :

```
docker compose down -v                # supprime les conteneurs et le volume Elasticsearch
rm -rf index index_1m                 # supprime les signatures (puis refaites les §1.3 et 4.2)
```

---

7. Ce qui est versionné et ce qui ne l'est pas

Le dépôt contient le **code**, la configuration Docker/Logstash, le rapport (`rapport/`) et ce guide. `.gitignore` exclut tout ce qui se reconstruit avec les commandes ci-dessus : `data/` (images et archives), `index/` et `index_1m/` (signatures), `.venv/`.

8. Remarque sur Docker

`docker-compose.yml` contient aussi un service `cbir` (le serveur dans un conteneur). Il est écrit mais **n'a pas été testé** : dans notre utilisation, Elasticsearch et Logstash tournent dans Docker et le serveur tourne avec Python sur la machine, comme décrit dans ce guide.